

Manual Técnico

TwinSight X500: visor WebGL para visualización técnica del Holybro X500 V2

Alexander Woodcock Salomón

Mayo 2026

Índice

1. Introducción	3
2. Convenciones canónicas	3
2.1. Taxonomía de piezas	3
2.2. Taxonomía de scripts	3
3. Verdad funcional del sistema	4
3.1. Flujo visible consolidado	4
4. Arquitectura runtime	4
4.1. Capas operativas	4
4.2. Bootstrap	4
4.3. Managers garantizados en runtime	5
4.4. Protección de input y convivencia UI/escena	6
5. Controladores UI y responsabilidades	6
5.1. Secuencia de interacción: Hero y entrada al visor	7
5.2. Secuencia de interacción: onboarding	7
5.3. Secuencia de interacción: selección y ficha inferior	7
5.4. Secuencia de interacción: tecla Esc	8
6. Flujos funcionales por modo	8
6.1. Modo Inspect	8
6.2. Modo Analyze	9
6.2.1. Filtros	9
6.2.2. Corte transversal	9
6.3. Modo Studio	10

7. Datos y normalización del modelo	10
7.1. Catálogo canónico	10
7.2. Catálogos modulares de fasteners	10
7.3. Pipeline final Blender a Unity	11
7.4. Saneamiento del import	11
7.5. De semántica a geometría	11
7.6. Inspección bajo demanda de fasteners	12
8. Subsistema de visualización y entorno	13
8.1. Modos de vista	13
8.2. Lógica de modo base	13
8.3. Panel de entorno	14
8.4. EnvironmentController	14
9. Subsistema de estado operativo del dron	14
9.1. Efectos coordinados por estado	15
9.2. Carga del sistema	15
10. Subsistema térmico híbrido	15
10.1. Alcance real	15
10.2. Pipeline térmico	16
10.3. Construcción del runtime térmico	16
10.4. Parámetros base del solver	16
10.5. Modelo de actualización térmica	16
10.5.1. Aproximación a equilibrio	17
10.5.2. Enfriamiento	17
10.5.3. Conducción entre nodos	17
10.5.4. Actualización final	17
10.6. Relación con el estado operativo	18
10.7. Traducción de temperatura a render	18
10.8. Patrones de superficie	19
10.9. Leyenda térmica	19
11. Estados funcionales del código	19
12. Tooling de editor	20
13. Módulos que no deben documentarse como activos	20
14. Build, validación y despliegue	20
15. Limitaciones vigentes del cierre	21

1. Introducción

Este manual técnico describe la arquitectura real de la build final documentada de TwinSight X500, el visor WebGL para el Holybro X500 V2. Su objetivo es explicar cómo se orquesta la aplicación en tiempo de ejecución, cómo se normaliza el modelo importado, cómo se conectan los modos visibles con los subsistemas internos y cuáles son los límites de la implementación entregada.

El documento no debe leerse como una promesa de todo el código presente en el repositorio. Por el contrario, diferencia explícitamente entre:

- funcionalidades expuestas en la interfaz final;
- funcionalidades implementadas pero ocultas;
- módulos legados o no integrados;
- trabajo futuro aún no consolidado.

2. Convenciones canónicas

2.1. Taxonomía de piezas

- **28 piezas canónicas:** entidades semánticas definidas en `x500v2_parts_data.json`.
- **30 anchors de escena:** 28 piezas canónicas más `x500v2_fastener_group` y `x500v2_misc_group`.
- **257 renderers/colliders:** granularidad técnica del import final auditado.
- **257 assets generados:** recursos `.asset` producidos para el binding y consumo runtime.

Esta distinción es crítica: una pieza canónica puede estar compuesta por múltiples renderers sin que eso altere la taxonomía académica del sistema.

2.2. Taxonomía de scripts

- **95** scripts runtime propios en `Assets/Scripts`.
- **103** scripts bajo `Assets` excluyendo `Editor/Tests`.
- **129** archivos `.cs` totales incluyendo editor y plugins.

3. Verdad funcional del sistema

- **Escena oficial:** `MainScene_Final`
- **Layout UI:** `MainLayout.uxml`
- **Control principal UI:** `UIManager.cs`
- **Saneamiento runtime del import:** `ImportedDroneRuntimeBinder.cs`
- **Documento de contraste funcional:** validación funcional final del proyecto, usada como referencia de cierre y verificación editorial.

3.1. Flujo visible consolidado

Hero ->Explore ->selección ->bottom sheet ->Inspect / Analyze / Studio

Este flujo es la base documental de la versión final. Toda referencia a paneles laterales, catálogos visibles, configuración general del sistema o herramientas no expuestas debe marcarse como oculta, legada o futura, nunca como parte del recorrido principal.

4. Arquitectura runtime

4.1. Capas operativas

1. **Presentación.** UI Toolkit, `MainLayout.uxml`, hojas de estilo y controladores de panel.
2. **Orquestación.** `UIManager`, `UIModeController`, `AppStateMachine` y enrutamiento de eventos.
3. **Servicios de escena.** selección, aislamiento, explosión, clipping, view modes, entorno, estado del dron y subsistema térmico.
4. **Datos.** `DronePartData`, assets generados, catálogo JSON canónico y grafo térmico.

4.2. Bootstrap

El arranque del visor combina dos mecanismos complementarios:

1. **`UIManager.EnsureManagers()`.** Garantiza la existencia de los servicios mínimos para operar la build final.
2. **`ImportedDroneRuntimeBinder`.** Repara la jerarquía del dron importado, reubica huérfanos, sincroniza anchors y reconstruye caches dependientes.

El orden funcional esperado puede resumirse así:

1. carga de `MainScene_Final`;
2. inicialización del `UIDocument`;
3. ejecución de `UIManager.InitializeUI()`;
4. creación o garantía de managers base;
5. inyección del `ImportedDroneRuntimeBinder` sobre `x500v2_Drone` si hace falta;
6. reconstrucción de servicios dependientes del modelo;
7. activación del flujo visible de interacción.

4.3. Managers garantizados en runtime

Módulo	Responsabilidad
<code>InputManager</code>	bloquea input 3D cuando el usuario interactúa con UI y evita click-through sobre la escena
<code>SelectionManager</code>	gestiona selección actual, doble clic y eventos de cambio de pieza
<code>FastenerRegistry</code>	resuelve en runtime la relación <code>instanceId ->family ->recipe</code> para fasteners
<code>FastenerInspectionManager</code>	emplaza solo el proxy seleccionado por detalle modular runtime y restaura el proxy al deseleccionar
<code>ExplodedViewManager</code>	controla separación del ensamblaje y filtros por categoría
<code>PartCatalogManager</code>	mantiene consulta runtime de piezas, aunque el catálogo no esté expuesto al usuario
<code>NotificationManager</code>	emite mensajes breves de estado y feedback operativo
<code>HotspotManager</code>	construye, inicializa y muestra hotspots
<code>ViewModeManager</code>	orquesta modos de visualización y modo base activo
<code>EnvironmentController</code>	aplica presets de skybox, luz y contraste general
<code>CrossSectionManager</code>	habilita clipping global y planos de corte
<code>PartVisibilityManager</code>	ocultar, mostrar, aislar y restaurar piezas o grupos
<code>DroneStateController</code>	estado operativo, carga, hélices, luces, partículas y audio opcional
<code>ThermalSimulation-Manager</code>	solver térmico reducido por componentes

Módulo	Responsabilidad
ThermalViewController	traduce temperaturas a parámetros de shader y leyenda visual

4.4. Protección de input y convivencia UI/escena

Un rasgo clave de la implementación final es el control explícito de conflicto entre UI y navegación 3D. Botones, sliders, *bottom sheet* y paneles registran eventos de puntero que activan `InputManager.InputBlocked` mientras el usuario interactúa con la interfaz. Este detalle evita que un arrastre sobre un slider, por ejemplo, termine rotando la cámara o seleccionando piezas accidentalmente.

5. Controladores UI y responsabilidades

Controlador	Función principal
UIManager	coordinador raíz; inicializa subcontroladores, enruta eventos y sincroniza subsistemas
UIHeroController	maneja Hero, retorno al inicio, acceso al repositorio y ayuda inicial
UIDetailsSheet	administra ficha inferior, peek bar, foco de cámara y contenido de selección
UIModeController	activa, desactiva y sincroniza <i>Inspect</i> , <i>Analyze</i> y <i>Studio</i>
InspectModeHandler	gestiona hotspots, isolate, power y carga
AnalyzeModeHandler	gestiona corte, explota y filtros con navegación de dos niveles
StudioModeHandler	contenedor visible del modo <i>Studio</i>
UIAnalyzePanel	controla modos de visualización y leyenda térmica
UIEnvironmentPanel	controla presets de entorno y sliders de iluminación
UICrossSectionPanel	controla ejes, inversión y segundo plano de corte
OnboardingController	muestra la ayuda inicial cuando el usuario entra al visor o la solicita
OnboardingAnimationView	renderiza las demos procedurales del onboarding en desktop y mobile mediante <i>Painter2D</i>

5.1. Secuencia de interacción: Hero y entrada al visor

El Hero no es un splash decorativo, sino el primer estado funcional del sistema. Desde `UIManager`, el controlador de Hero emite eventos que:

- habilitan la inicialización visible de hotspots cuando el usuario entra al visor;
- muestran la guía de onboarding la primera vez;
- permiten volver al Hero, cerrar modos activos y restablecer la vista principal.

5.2. Secuencia de interacción: onboarding

El onboarding visible no depende de video ni de media embebida. Su arquitectura actual combina dos piezas:

- `OnboardingController`, que decide cuándo mostrar la ayuda, qué tarjeta está activa, qué *copy* debe aparecer y cómo cambia la plataforma entre desktop y mobile;
- `OnboardingAnimationView`, que dibuja cada demostración mediante `Painter2D` como una escena procedural de actor, objetivo y respuesta del sistema.

Este diseño permite mantener la ayuda sincronizada con la UI real. Cada tarjeta del onboarding reutiliza el mismo lenguaje visual de cursores, ripples, sliders, botones y mini iconos del producto, en lugar de depender de GIFs o secuencias externas difíciles de actualizar.

5.3. Secuencia de interacción: selección y ficha inferior

Cuando `SelectionManager` emite un `PartSelectedEvent`, `UIManager` delega el contenido a `UIDetailsSheet`. A partir de ahí ocurre lo siguiente:

1. se actualiza el nombre visible de la pieza en la barra superior e inferior;
2. se llenan campos editoriales desde `DronePartData`;
3. si la ficha está cerrada, aparece la barra de previsualización;
4. si la ficha se abre, la cámara puede recentrar la selección;
5. si no hay selección, la interfaz vuelve a `SELECT A PART`.

La selección visible ya no debe leerse como un único nivel plano. El sistema diferencia:

- **pieza madre**, cuando la interacción se resuelve sobre el conjunto principal;
- **subpieza**, cuando el usuario profundiza sobre una geometría interna del conjunto activo;
- **grupo de hotspot**, cuando la entrada llega desde un pin y representa una lectura agrupada del sistema.

Por esa razón, un clic de fondo puede retroceder un nivel de lectura y un doble clic puede activar aislamiento del nivel actual. Esta jerarquía es importante porque la ficha, la cámara, el isolate y algunos paneles de análisis no siempre trabajan sobre la misma granularidad semántica.

La ficha inferior contiene reglas de presentación importantes:

- `PartPower` muestra vatios solo si el dato existe;
- `PartTemp` muestra temperatura operativa si es mayor que cero, o N/A;
- la dificultad se renderiza como calificación visual por estrellas;
- selecciones provenientes de hotspots pueden presentar títulos y descripciones grupales.

Para fasteners, `SelectionManager` promueve cualquier clic sobre la geometría modular al root completo del fastener antes de publicar el evento de selección. Esto evita aislamientos parciales por submallá y asegura que el color de *hover* o selección se reaplique correctamente cuando `FastenerInspectionManager` sustituye el proxy por detalle modular runtime.

A nivel de aislamiento, `PartVisibilityManager` separa varios casos: fastener individual, pieza madre, subpieza y grupo de hotspot. El fastener individual se muestra como unidad completa; la pieza madre puede conservar fasteners reconciliados mediante `parentCanonicalPartId`; la subpieza estrecha el contexto al nivel más fino disponible; y el grupo de hotspot permite una lectura agregada cuando el usuario entra desde un pin en lugar de hacerlo desde geometría directa.

5.4. Secuencia de interacción: tecla Esc

`UIManager` define una prioridad explícita para `Esc`:

1. cerrar la ficha inferior si está abierta;
2. si la ficha no está abierta, delegar al `UIModeController` para cerrar subpaneles o desactivar el modo actual;
3. no realizar acciones adicionales si no hay estados abiertos.

6. Flujos funcionales por modo

6.1. Modo Inspect

`InspectModeHandler` controla cuatro elementos visibles:

- **Hotspots.** Inician activos. Su visibilidad se conmuta desde el gestor de hotspots del sistema.
- **Isolate.** Solicita aislamiento a través de `UIManager`, que a su vez resuelve si el aislamiento es de pieza madre, subpieza, fastener individual o grupo de hotspot.
- **Power.** Conmuta el estado de `DroneStateController`.
- **Load.** Envía un valor normalizado a `DroneStateController.SetLoadCommand()` para actualizar la carga operativa.

El handler también sincroniza la etiqueta de estado con los estados OFF, STARTING, IDLE, FLYING y SHUTDOWN. La herramienta de medición existe en código, pero su botón permanece oculto en la UI final, por lo que no debe documentarse como función visible de usuario.

6.2. Modo Analyze

AnalyzeModeHandler usa una navegación de dos niveles:

- **Nivel 1:** grilla principal de tarjetas.
- **Nivel 2:** subpanel de `Cross Section` o subpanel de `Filter`.

La vista explosionada se resuelve de forma distinta: no abre un panel aparte, sino un deslizador en línea debajo de la grilla principal. El handler conserva el último valor distinto de cero para restaurarlo si la herramienta se vuelve a activar.

6.2.1. Filtros

Los filtros activos iniciales corresponden a:

- `SkeletonAirframe`
- `PropulsionSystem`
- `Avionics`
- `SensorsComms`
- `PowerDistribution`
- `Fasteners`

Si el usuario apaga la última categoría visible, el handler restablece el conjunto por defecto para evitar un estado completamente vacío. Además, el controlador acepta modo exclusivo con doble clic sobre la categoría.

6.2.2. Corte transversal

`UICrossSectionPanel` soporta hasta dos planos simultáneos. La lógica real es:

- el panel inicia con el eje Y activo;
- el usuario puede activar hasta dos ejes;
- si selecciona un tercer eje, se elimina el más antiguo mediante estrategia FIFO;
- cada plano puede invertirse de forma independiente;
- el segundo bloque de controles solo aparece cuando hay dos ejes activos y el modo combinado no está habilitado;

- si todos los ejes se desactivan, `CrossSectionManager` deshabilita el corte;
- el corte persiste al volver desde el subpanel y solo se limpia por deselección total de ejes o `RESET VIEW`.

6.3. Modo Studio

`Studio` combina dos subsistemas:

1. `UIAnalyzePanel`, que gobierna los modos de visualización.
2. `UIEnvironmentPanel`, que gobierna presets de entorno e iluminación.

En la UI final, `Studio` no solo cambia modos visuales. También controla ciclos de entorno y tres deslizadores de iluminación visibles para el usuario: rotación de luz, intensidad sobre el objeto y tono del fondo.

7. Datos y normalización del modelo

7.1. Catálogo canónico

`x500v2_parts_data.json` es la fuente semántica oficial del proyecto. De este archivo derivan:

- el conteo canónico de piezas;
- la categorización funcional;
- parte de los datos que consume `DronePartData`;
- la narrativa académica y documental del sistema.

7.2. Catálogos modulares de fasteners

La implementación Unity incorpora una capa específica para tornillería y piezas de fijación, separada del catálogo canónico de piezas principales. Esta capa se materializa en cuatro archivos:

- `holybro_fastener_families.json`: define familias modulares con métrica, longitud, material, receta visual y referencia CAD/Holybro.
- `holybro_fastener_instances.json`: registra instancias presentes en `MainScene_Final` con `instanceId`, pose local y anchor canónico inferido.
- `holybro_fastener_reconciliation.json`: documenta alias, overrides semánticos y discrepancias entre escena y dataset sincronizado.
- `holybro_parent_subpieces.json`: fija la relación pieza madre → subpiezas documentadas → fasteners configurados y se replica en `subComponentNames` dentro de los assets generados.

En el estado documentado de cierre, estos catálogos registran **20 familias, 168 instancias y 9 entradas de reconciliación**. Los tornillos inspeccionables usan detalle modular importado bajo demanda; tuercas, standoffs, grommets y otros elementos no-tornillo conservan su geometría original. El contrato `familyId / instanceId` queda estable para futuras sustituciones visuales sin alterar la lógica runtime.

7.3. Pipeline final Blender a Unity

El reimport final del modelo optimizado se define como una exportación manual desde Blender 5.0.1. La escena runtime no se compone solo de instancias: debe incluir simultáneamente `BAKE_MASTERS_LOW`, `ASSEMBLY_INSTANCES_LOW`, `PRIMITIVE_FASTENER_MASTERS` y `PRIMITIVE_FASTENER_INSTANCES`. Excluir los masters provocaría pérdida física de piezas del dron.

El paquete de texturas recomendado para Unity queda compuesto por `X500_BaseColor_4K.png`, `X500_Normal_Final_4K.png` y `X500_Mask_4K.png`. La máscara compacta usa los canales `R=AO`, `G=Roughness`, `B=Curvature` y `A=Metallic`. Para compatibilidad con URP Lit estándar puede generarse también `X500_MetallicSmoothness_4K.png`.

La importación posterior debe apoyarse en un manifest de Blender que reporte transformaciones, bounds, roles runtime y candidatos de pieza madre para fasteners. Si un fastener no puede asociarse con confianza a una pieza madre, permanece en revisión y no se asigna automáticamente.

7.4. Saneamiento del import

`ImportedDroneRuntimeBinder` ejecuta tareas de estabilización sobre el modelo importado:

- resuelve el root del dron;
- construye el mapa de anchors por identificador;
- reubica objetos huérfanos o mal anidados;
- crea o reutiliza grupos sintéticos para tornillería y elementos misceláneos;
- garantiza `FastenerRegistry` y `FastenerInspectionManager`;
- sella `fastenerFamilyId` y `fastenerInstanceId` sobre fasteners detectados;
- reconstruye caches de managers dependientes;
- dispara eventos para regeneración de hotspots y auditorías.

7.5. De semántica a geometría

El sistema trabaja con varias capas de representación:

1. **Semántica:** pieza canónica y sus metadatos.

2. **Escena:** anchor o transform padre usado para binding.
3. **Geometría:** renderers y colliders individuales.
4. **Visualización:** materiales, property blocks, clipping y modos de vista.

Esta separación permite que la UI describa un componente a nivel conceptual aunque el import final lo represente con múltiples submallas.

7.6. Inspección bajo demanda de fasteners

La optimización de fasteners no se resuelve cargando mallas densas todo el tiempo. El comportamiento implementado es:

1. la escena permanece en reposo con proxies ligeros;
2. al seleccionar un fastener, `FastenerRegistry` resuelve su metadata técnica;
3. `FastenerInspectionManager` puede activar un conjunto acotado de detalles modulares según contexto;
4. `FastenerBuilder` instancia el detalle modular para el fastener seleccionado, para un fastener aislado aunque ya no permanezca seleccionado y para los fasteners asociados a una pieza madre aislada;
5. al salir del contexto correspondiente, cada detalle se destruye y su proxy original se restaura.

Esta decisión mantiene bajo el costo visual de la escena actual y, al mismo tiempo, permite inspección detallada sin cargar toda la tornillería de alta densidad al iniciar la experiencia.

En paralelo, `HighlightSystem`, `MaterialController` y `PartVisibilityManager` refrescan sus objetivos visuales y su scope de aislamiento para que el fastener siga respondiendo como unidad completa durante *hover*, selección e isolate.

La cámara también deja de usar una distancia fija de inspección. `OrbitCameraController` calcula el enfoque a partir de los bounds reales de la selección y reduce el *near clip* para permitir acercamientos útiles sobre fasteners y piezas pequeñas.

Además, la ventana de zoom deja de ser global. La sensibilidad y los límites de acercamiento se recalculan por selección para evitar que un rango válido para el dron completo resulte tosco o excesivo sobre una pieza de fijación pequeña.

Ese recálculo ya no depende del isolate: una selección activa dentro del ensamblaje completo también puede estrechar la ventana de zoom para habilitar acercamientos más finos sin ocultar el contexto restante.

En el mismo controlador, los factores de `pan` y `orbit` pasan a escalar con la distancia y el contexto efectivo de inspección. Con ello se evita que una sensibilidad correcta para el dron completo resulte excesiva cuando el usuario analiza un fastener o una subpieza muy pequeña.

La iteración más reciente sustituye la respuesta lineal de ambos factores por curvas separadas: `pan` cae con mayor rapidez en escalas pequeñas para priorizar precisión de centrado, mientras `orbit` conserva una respuesta angular algo más alta para no entorpecer la inspección visual.

Para evitar bloqueos al deshacer zoom, el controlador ya no usa ciegamente el último `target` enfocado como contexto adaptativo. La prioridad queda definida como selección activa, luego aislamiento activo y, en ausencia de ambos, el contexto base del modelo. Además, la ventana adaptativa conserva un margen mínimo de alejamiento para que una inspección cercana no cierre por completo la salida hacia vistas más generales.

Como refinamiento final, la curva de `pan` para escalas pequeñas se endurece aún más que en la iteración previa. El objetivo es que el usuario pueda recentrar un fastener aislado con desplazamientos cortos sin expulsarlo fácilmente del encuadre.

8. Subsistema de visualización y entorno

8.1. Modos de vista

`UIAnalyzePanel` conoce internamente los modos:

- `Realistic`
- `XRay`
- `Blueprint`
- `SolidColor`
- `Wireframe`
- `Ghosted`
- `Thermal`

Sin embargo, la UI final solo expone directamente `X-Ray`, `Solid View / Solid Color` y `Thermal`. `Realistic` opera como modo base, mientras que `Blueprint` se activa desde el ciclo de presets de `Studio`. En la interfaz, `Blueprint` no es una tarjeta independiente de shader, pero sí forma parte del flujo publicado como preset de visualización técnica.

8.2. Lógica de modo base

La distinción entre modo base y modo activo es importante:

- si el usuario pulsa un modo visual que ya está activo, el sistema regresa al modo base;
- normalmente el modo base es `Realistic`;
- si el preset `Blueprint` se activa desde el panel de entorno, el modo base pasa temporalmente a `Blueprint`.

8.3. Panel de entorno

`UIEnvironmentPanel` implementa tres ciclos:

- `Studio` → `Studio Light` → `Blueprint`
- `Day` → `Night` → `Sunset`
- `White` → `Grey` → `Black` → `Yellow` → `Orange` → `Green` → `Blue` → `Purple` → `Red`

El controlador actualiza dinámicamente la etiqueta de cada tarjeta para reflejar el preset vigente y, cuando corresponde, restablece el modo base al salir de `Blueprint`. En paralelo, coordina los tres sliders visibles de `Lighting Controls`: `Light Rotation`, `Light Intensity` y `Background Tone`.

8.4. EnvironmentController

`EnvironmentController` no opera con fondos planos simples. Usa el shader `AnimatedGradientSkybox` y define presets mediante un conjunto de parámetros:

- color superior e inferior del gradiente;
- color e intensidad de la luz direccional;
- rotación y pitch de la luz;
- escala del gradiente;
- dither;
- pulso opcional;
- activación de grilla para `Blueprint`.

El controlador interpola transiciones entre presets y emite `OnLightBackgroundChanged` cuando detecta que el centro del gradiente alcanza suficiente luminancia para requerir UI adaptativa. `UIManager` consume ese evento para alternar clases visuales y mejorar legibilidad. Esto significa que el control de iluminación del fondo no es un simple color plano. El sistema modifica la percepción general del entorno mediante skybox animado, gradiente, contraste, intensidad de luz y adaptación de la propia interfaz cuando el fondo se vuelve muy claro.

9. Subsistema de estado operativo del dron

`DroneStateController` articula la simulación operativa ligera de la escena. Sus estados son:

- `Off`

- StartingUp
- Idle
- Flying
- ShuttingDown

9.1. Efectos coordinados por estado

El cambio de estado afecta:

- velocidad de hélices;
- iluminación de estado;
- partículas;
- desplazamiento de hover;
- clips de audio, si existen recursos asignados.

9.2. Carga del sistema

La carga se expresa como un valor normalizado $u \in [0, 1]$ enviado por `SetLoadCommand`. La lógica relevante es:

- un umbral aproximado de 0.35 separa reposo operativo y vuelo;
- el sistema mantiene un piso armado de carga aun con valores bajos cuando el dron está encendido;
- el valor final de carga alimenta el estado térmico del sistema.

10. Subsistema térmico híbrido

10.1. Alcance real

El sistema térmico implementado no es un solver FEA ni una simulación termodinámica de alta fidelidad. Su alcance real es el de una **simulación híbrida heurística por componentes**, diseñada para:

- comunicar zonas críticas;
- propagar calor de forma creíble entre componentes vinculados;
- responder al estado operativo del dron;
- alimentar una visualización térmica en tiempo real adecuada para WebGL.

10.2. Pipeline térmico

```
DroneStateController ->ThermalSimulationManager  
->ThermalViewController ->shader/UI
```

10.3. Construcción del runtime térmico

`ThermalSimulationManager` recorre los `ExplodablePart` presentes en escena y construye un conjunto de nodos térmicos. Para cada nodo resuelve:

- identificador de pieza;
- clase de fuente térmica;
- temperatura actual y temperatura de equilibrio;
- temperatura mínima, de hover y pico;
- tiempo de calentamiento;
- tasa de enfriamiento;
- exposición;
- peso de la fuente;
- escala de conducción;
- criticidad térmica.

Luego construye enlaces térmicos mediante dos estrategias:

- **grafo autoritativo**, si existe un `ThermalContactGraphAsset` válido;
- **enlaces fallback**, si el grafo no está disponible.

10.4. Parámetros base del solver

- temperatura ambiente por defecto: **20 °C**;
- escala temporal de simulación: **4x**;
- frecuencia objetivo del solver: **20 Hz**;
- actualización visual térmica: **12 Hz**.

10.5. Modelo de actualización térmica

En cada paso de simulación, el sistema combina tres contribuciones: acercamiento a equilibrio, enfriamiento hacia ambiente y conducción con vecinos.

10.5.1. Aproximación a equilibrio

Primero se calcula un factor de mezcla exponencial:

$$\alpha = 1 - e^{-\Delta t/\tau}$$

donde Δt es el paso de simulación y τ representa el tiempo característico de calentamiento.

La contribución de la fuente se calcula como:

$$\Delta T_{\text{fuente}} = (T_{\text{eq}} - T) \alpha w_s$$

donde T_{eq} es la temperatura de equilibrio para la carga actual, T la temperatura presente y w_s el peso de la fuente térmica.

10.5.2. Enfriamiento

El enfriamiento simplificado se expresa como:

$$\Delta T_{\text{enfriamiento}} = (T_{\text{amb}} - T) k_c e \Delta t$$

donde T_{amb} es la temperatura ambiente, k_c la tasa de enfriamiento y e el factor de exposición del componente.

10.5.3. Conducción entre nodos

Cada enlace térmico agrega un intercambio:

$$\Delta T_{A \leftrightarrow B} = (T_B - T_A) G_{\text{eff}} \Delta t$$

donde G_{eff} es la conductancia efectiva del vínculo.

Cuando existe un grafo térmico autoritativo, la conductancia base parte de:

$$G_{\text{base}} = \max \left(0,001, s \cdot \frac{A}{L} \right)$$

donde s es la escala de conducción del enlace, A el área de contacto y L la longitud del trayecto. Luego el sistema la ajusta con escalas de material, criticidad y parámetros propios de los nodos conectados.

10.5.4. Actualización final

La temperatura nueva del nodo se calcula como:

$$T_{\text{nuevo}} = \text{clamp} \left(T + \Delta T_{\text{fuente}} + \Delta T_{\text{enfriamiento}} + \sum \Delta T_{\text{vecinos}}, T_{\text{mín}}, T_{\text{máx}} \right)$$

Este modelo es coherente con la implementación real y suficientemente ligero para ejecutarse en runtime dentro del visor WebGL.

10.6. Relación con el estado operativo

La carga efectiva depende del estado del dron:

- `Off` y `ShuttingDown` llevan la carga a cero;
- `StartingUp` establece una carga baja inicial;
- `Idle` aplica un piso operativo mínimo;
- `Flying` impone una carga mínima asociada a vuelo.

Con esta lógica, la simulación térmica responde de manera coherente a los controles visibles de `Power` y `Load`.

Las hélices se modelan como elementos pasivos de baja respuesta térmica dentro de la visualización heurística. No se reportan como fuente caliente primaria porque, en el alcance del prototipo, no concentran disipación eléctrica ni generación térmica interna comparable con motores, batería o electrónica. Cualquier activación visual debe leerse como acoplamiento aproximado con el conjunto motor-hub, no como medición calibrada de temperatura en las palas.

10.7. Traducción de temperatura a render

`ThermalViewController` no dibuja simplemente un color uniforme por pieza. Primero resuelve un valor normalizado:

$$n = \text{InverseLerp}(T_{\text{disp,min}}, T_{\text{disp,max}}, T)$$

con $T_{\text{disp,min}} = 20^\circ\text{C}$ y $T_{\text{disp,max}} = 100^\circ\text{C}$ en la configuración actual visible.

Luego construye una banda térmica:

$$n_{\text{mín}} = \text{clamp01}(n - b), \quad n_{\text{máx}} = \text{clamp01}(n + b)$$

donde b es el ancho medio de banda, distinto para piezas críticas, activas o pasivas.

Finalmente, el controlador envía al shader propiedades por `MaterialPropertyBlock`, entre ellas:

- rango térmico mínimo y máximo;
- hotspot local;
- dirección de propagación;
- *spread*;
- enfriamiento de bordes;
- variación base;
- propagación.

10.8. Patrones de superficie

La capa visual clasifica la distribución térmica de cada pieza en patrones como:

- **radial**, típico de motores, batería o núcleo electrónico;
- **axial**, típico de brazos, ESC o rieles;
- **uniforme**, típico de placas y ciertos elementos estructurales.

Esta clasificación puede provenir de un perfil explícito de superficie térmica o inferirse desde la pieza canónica y su geometría.

10.9. Leyenda térmica

La leyenda del modo `Thermal` se enlaza desde `ThermalViewController` con tres elementos de UI:

- etiqueta mínima;
- etiqueta máxima;
- gradiente visual generado en runtime.

El gradiente recorre un espectro que va de tonos fríos oscuros a zonas claras de alta energía. Su función es interpretativa, no metrológica.

11. Estados funcionales del código

Estado	Contenido
Expuesto en UI final	Hero, selección de piezas, <i>bottom sheet</i> , Inspect, Analyze, Studio, presets de entorno incluyendo Blueprint, modos X-Ray/Solid View/Thermal, leyenda térmica
Implementado pero oculto	medición, modos Wireframe, Ghosted, Realistic como botón directo, partes avanzadas del flujo térmico
Código legado o no integrado	PartCatalogUI, SettingsPanel, LoadingController, partes de EnhancedInfoPanel, suite de ensamblaje y anotaciones
Trabajo futuro	métricas cuantitativas finales, reemplazo opcional de activos CAD/import, audio final si llega a integrarse y validarse

12. Tooling de editor

- **ProjectSetupWizard.** Asistente de estructura inicial de escena.
- **ImportedDroneCoverageAudit.** Auditoría de cobertura entre jerarquía importada y catálogo canónico.
- **ThermalContactGraphBuilderWindow.** Herramienta offline para generación del grafo térmico autoritativo.
- **WebGLBuildFixer y utilidades Antigravity.** Ajustes de configuración y build.

13. Módulos que no deben documentarse como activos

Este manual excluye como módulos activos finales a:

- `ViewModeToolbar`
- `WebGLOptimizer`
- `ScreenshotManager`
- `KeyboardShortcuts`
- `PerformanceMonitor`
- `RuntimeConsole`
- `SceneTransitionManager`
- `TooltipSystem`

14. Build, validación y despliegue

1. abrir el proyecto con la versión de Unity documentada en el repositorio;
2. verificar que la escena activa sea `MainScene_Final`;
3. asegurar que el binding runtime del dron y los managers mínimos estén presentes;
4. generar build WebGL;
5. validar jerarquía, cobertura, flujo visible y leyenda térmica antes de generar evidencias de evaluación;
6. registrar sesiones del profiler interno y conservar trazabilidad de los resultados cuantitativos de usabilidad/rendimiento reportados en el informe.

15. Limitaciones vigentes del cierre

- el reemplazo por FBX runtime final con masters, instancias, fasteners, hotspots, filtros y hélices queda como mejora de pipeline si se cambia la escena publicada;
- las texturas/materiales finales deben documentarse como parte de la build publicada o mantenerse como mejora visual futura;
- los efectos de audio no deben presentarse como parte del cierre funcional verificado si no existen assets finales;
- la herramienta de medición/profiler se usa para capturar KPIs internos y debe manejarse como herramienta de validación, no como flujo principal de usuario;
- el subsistema térmico es coherente con la app real, pero debe documentarse como simulación híbrida heurística, no como FEA;
- los resultados académicos de KPIs, SUS, NASA-TLX Raw y tareas se reportan en el capítulo 5 del informe.